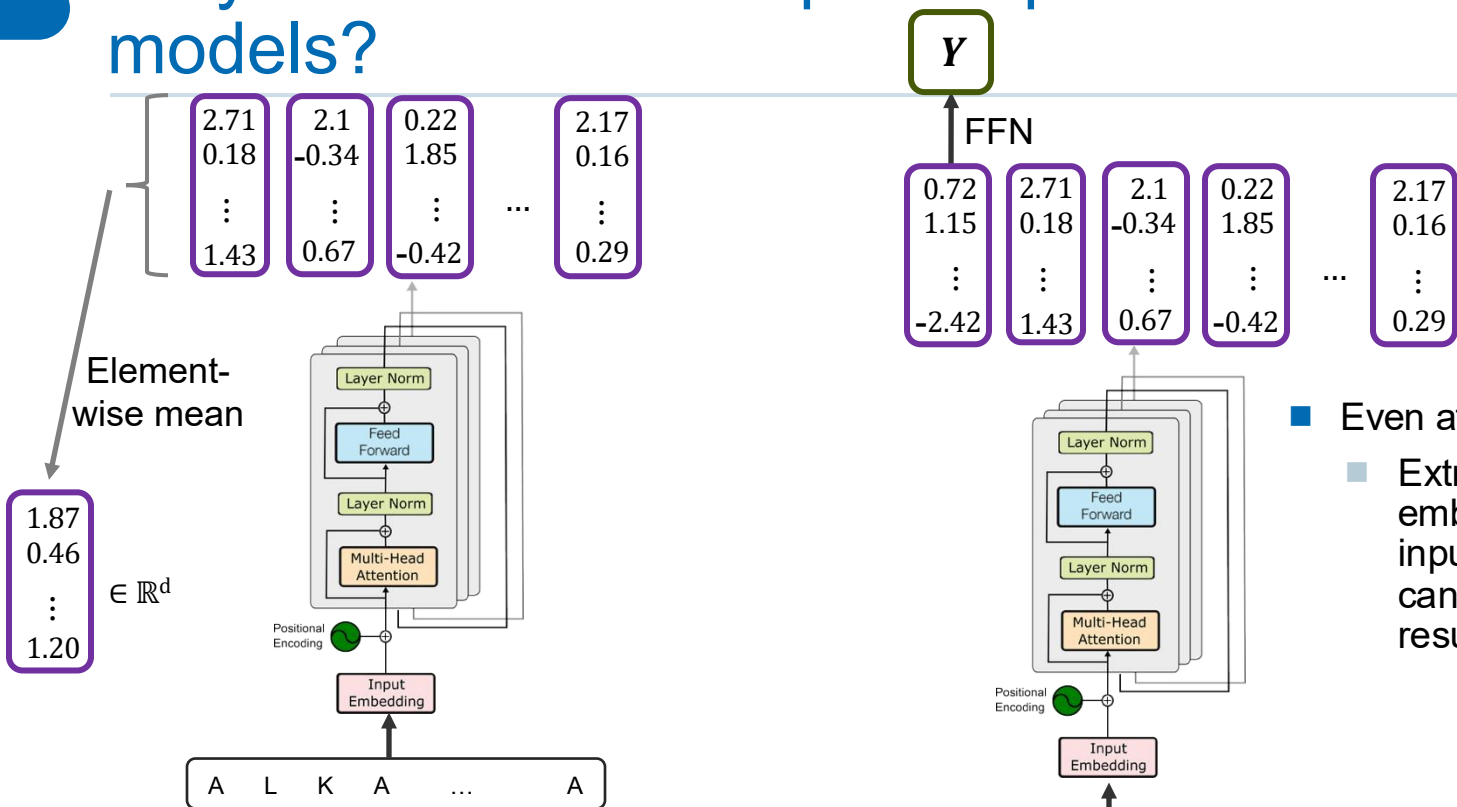


Gradient Boosting Decision Tree Models

Why do we need a separate prediction models?



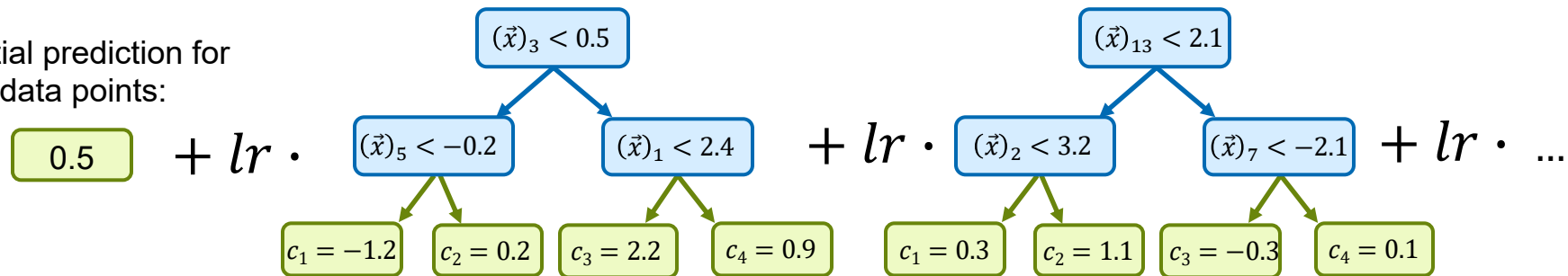
- Even after fine-tuning:
 - Extracting updated cls-embedding & using these as input for a separate model, can lead to better prediction results

- After self-supervised training, we need a prediction model for the obtained embeddings

What are Gradient Boosting Decision Tree Models?

- Let's assume we have input vectors $\vec{x} \in \mathbb{R}^d$. How do we get a prediction $y \in \mathbb{R}$?

Initial prediction for
all data points:



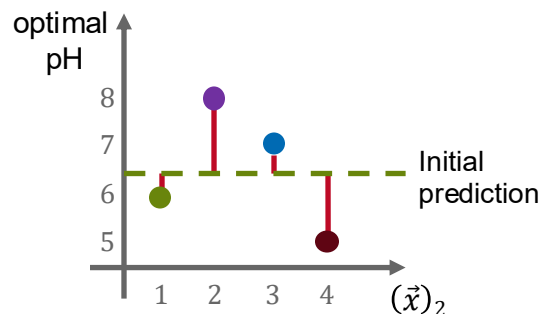
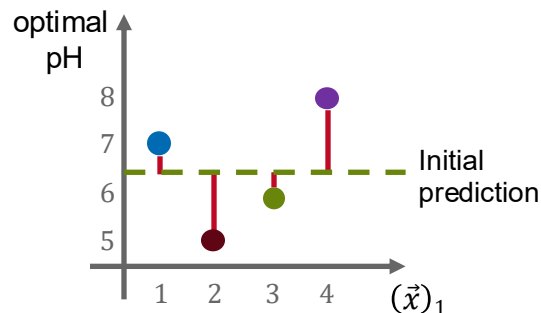
- Example:

- learning rate $lr = 0.3$
- We are in leaf c_1 for tree 1
- We are in leaf c_3 for tree 2

$$y_{pred} = 0.5 + 0.3 \cdot (-1.2) + 0.3 \cdot (-0.3) + \dots$$

Building Trees During Training

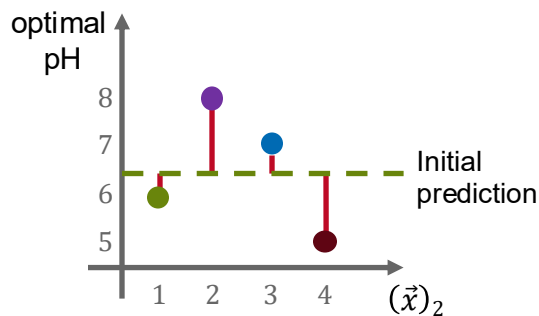
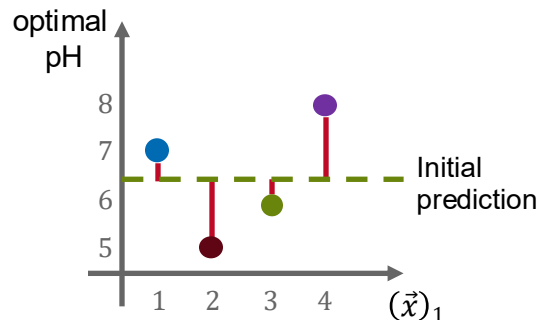
- Example training data: $\mathcal{D} = \{(\vec{x}_1, y_1), \dots, (\vec{x}_n, y_n)\}$, $\vec{x}_i \in \mathbb{R}^d, y_i \in \mathbb{R}$ with $n = 4, d = 2$.



- Initial prediction for all data points: 6.5
- Find decision rule $(\vec{x})_i < a$ that maximizes similarity of prediction errors e_i in two new subgroups
- Calculate similarity score for the two subgroups: $S = \frac{(\sum_i e_i)^2}{n}$
- $S_{Root} = \frac{(0.5 - 1.5 - 0.5 + 1.5)^2}{4} = 0$

Data point	Prediction error e_i
●	0.5
●	-1.5
●	-0.5
●	1.5

Building Trees During Training (2)



- Similarity score:

- $$S = \frac{(\sum_i e_i)^2}{n}$$

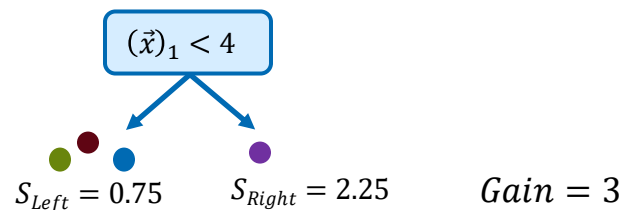
- $$S_{Root} = \frac{(0.5 - 1.5 - 0.5 + 1.5)^2}{4} = 0$$

- Aim:** Find decision rule $(\vec{x})_i < a$ that splits the dataset in a left and a right group that maximizes the gain:

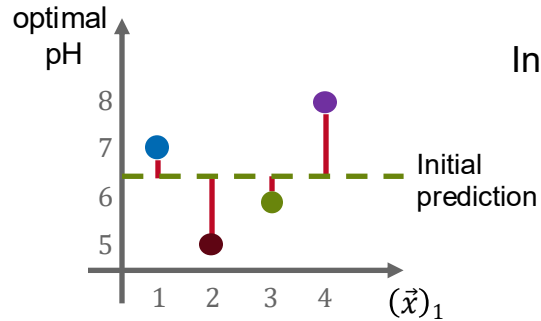
$$Gain = S_{Left} + S_{Right} - S_{Root}$$

- We test all possible values for i and a and select the combination with the highest *Gain*

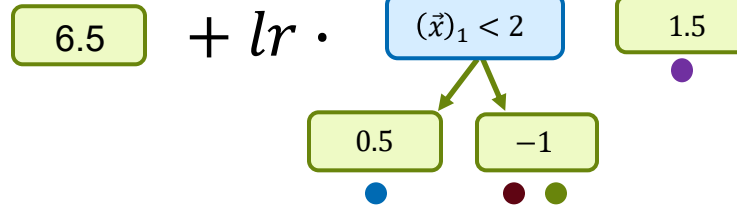
Data point	Prediction error e_i
●	0.5
●	-1.5
●	-0.5
●	1.5



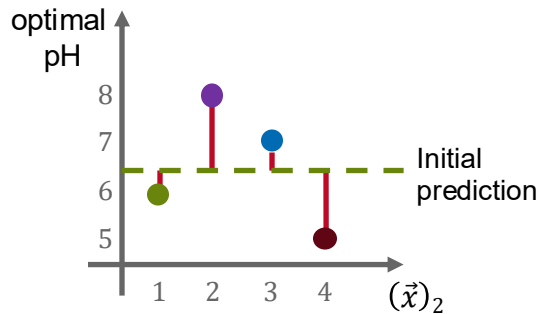
Building Trees During Training (3)



Initial prediction:



Data point	Prediction error e_i
●	0.5
●	-1.5
●	-0.5
●	1.5



For $lr = 0.5$:

Data point	New prediction	New prediction error
●	$6.5 + 0.5 \cdot 0.5 = 6.75$	0.25
●	$6.5 + 0.5 \cdot (-1) = 6.00$	-1.00
●	$6.5 + 0.5 \cdot (-1) = 6.00$	0.00
●	$6.5 + 0.5 \cdot 1.5 = 7.25$	0.75

Most Important Hyperparameters

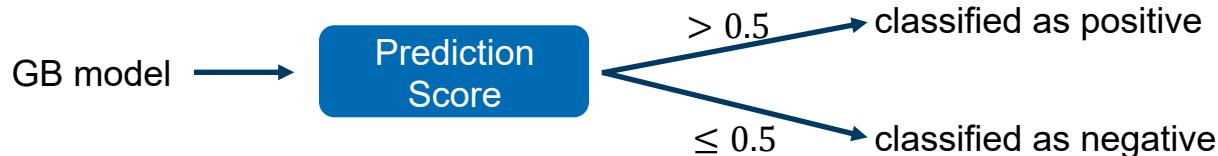
- *learning_rate*
 - Specifies how much weight the prediction of each new tree gets
 - Range: (0,1]
 - Default: 0.3
- *reg_lambda* and *reg_alpha*
 - Regularization coefficients that
 - penalize large predictions in the leaves
 - weaken the influence of branches that contain only few data points
 - Range: $[0, \infty]$ for both
 - Default: 1 for *reg_lambda* and 0 for *reg_alpha*
- *max_depth*
 - Defines to maximum allowed depth for each tree.
 - Default: 6

Most Important Hyperparameters (2)

- *n_estimators*
 - Defines the numbers of tree that will be build in the gradient boosting model
 - Range: Any positive integer
- *min_child_weight*
 - Tells the model when to stop splitting tree nodes when:
 - number of data points in this node is too small (regression task)
 - most data points in this node belong to one class (classification task)
 - Range: Any positive integer
 - Default: 1
- For additional hyperparameters see the XGBoost documentation:
<https://xgboost.readthedocs.io/en/stable/parameter.html>

Gradient Boosting for Binary Classification

- Gradient Boosting for binary classification tasks works very similar as for regression tasks
- Model output is a prediction score between 0 and 1:



- With each tree we try to predict the prediction error:
 - $Error = True\ class\ label - Prediction\ score$
- Differences to regression prediction task:
 - Calculation of similarity scores that are required for splitting tree nodes
 - Calculating the prediction values in the leaves

Gradient Boosting for Binary Classification (2)

- The model predicts a $\log(\text{odds})$ score s

- $\text{odds} = \frac{\text{likelihood of positive class}}{\text{likelihood of negative class}} = \frac{p}{1-p}$

- Converting $\log(\text{odds})$ score s to a probability score by applying the logistic function

$$\text{sigmoid}(s) = \frac{e^s}{1 + e^s} = p$$

- Initial prediction: $\log(\text{odds})$ based on the training data points:

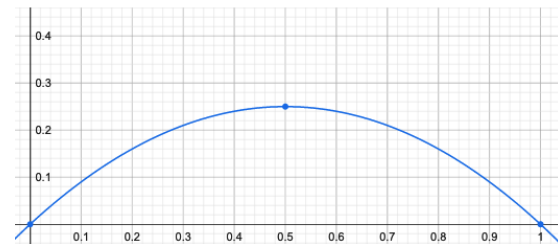
$$\frac{\# \text{positive class labels}}{\# \text{negative class labels}}$$

- With each tree we try improve the prediction error:

- $\text{Error} = \text{True class label} - \text{predicted probability}$

- Computing predictions for all data points in the same leaf:

$$\frac{\sum_i \text{Error}_i}{\sum_i \text{predicted probability}_i \times (1 - \text{predicted probability}_i)}$$



Python

Python

Python

Python

hhu.de

Plotting XGBoost trees

```
from xgboost import plot_tree
import matplotlib.pyplot as plt
plot_tree(model, num_trees=0)
plot_tree(model, num_trees=1)
plt.show()
```

✓ 0.5s

Python

